MATERI WEEK 06 - Deadlock

A. PRACTICUM 6.1 :

1. Prak6-1 : Critical Region & Race Condition in Threads

a. prak6-1_NIM.c

Tulis kode dibawah dan lakukan kompilasi pada program ini dan jalankan !

```
// SAVE DENGAN NAMA : prak6-1_NIM.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NITTERS 10000000 // Jumlah iterasi yang akan dilakuka
n oleh setiap thread
void *count(void *arg);
unsigned int cnt = 0; // Variabel global yang akan digunakan
sebagai counter bersama
// variabel cnt ini akan diakses oleh kedua thread

int main() {
    // Deklarasi thread
    pthread_t tid1, tid2;

    // Membuat thread
    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);
    /*
```

```
    FORMAT FUNGSI : pthread_create(thread_id, attr, function,
arg);
    1) thread_id :  Biasanya berupa alamat dari variabel thre
ad
    2) attr :  (DEFAULT) NULL, untuk menggunakan atribut defa
ult
    3) function :  Fungsi yang akan dijalankan oleh thread
    4) arg :  Argumen yang akan diteruskan ke fungsi thread,
jika tidak ada bisa diisi NULL
    */

    // Menunggu kedua thread selesai
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    // Mengecek nilai counter setelah kedua thread selesai
    if(cnt != (unsigned)NITTERS * 2) {
        printf("Wrong.. cnt = %d\n", cnt);
    } else {
        printf("Ok.. \tcnt = %d\n", cnt);
    }
    exit (0);
}

// Function yang akan dijalankan oleh kedua thread untuk mela
kukan increment pada cnt
void *count(void *arg) {
    int i;

    for(i = 0; i < NITTERS; i++) {
        cnt++;
    }
}
```

COMPILE DENGAN PERINTAH : `gcc prak6-1_NIM.c -o prak6-1_NIM -pthread`

b. PENJELASAN

1. Sebelum menjalankan program ini, tentukan berapa seharusnya final value tersebut? Setelah menjalankan program ini, apa final value dari cnt?

Nilai yang kita harapkan tentunya adalah 20000000, hal ini dikarenakan secara matematis terdapat 2 thread yang masing-masing melakukan iterasi sebanyak NITTERS, secara matematis juga nilai akhir cnt seharusnya yaitu: $10.000.000 * 2 = 20.000.000$

Namun, realitanya hampir selalu bukan 20.000.000, nilainya akan acakdan berubah-ubah setiap kali program dijalankan. Program ini akan mencetak pesan "Wrong... cnt = ...". Kita dapat mengambil kesimpulan bahwa output program ini Non-Deterministik / Tidak Pasti.

2. Tentukan critical region pada program ini, dan jelaskan race condition yang terjadi pada program ini!

a. **Critical Region** adalah bagian dari kode program yang mengakses shared resource, dimana akses tersebut tidak boleh diakses / dilakukan oleh lebih dari 1 thread pada saat yang bersamaan.

Dalam kode diatas, critical regionnya adalah :

```
cnt++;
```

Meskipun hanya sebagai 1 baris, di tingkat instruksi mesin, operasi ini terdiri dari 2 langkah yaitu:

- Load: Membaca nilai `cnt` dari memori ke register CPU
- Increment: Menambahkan nilai 1 di dalam register
- Store: Menuliskan kembali nilai dari register ke memori `cnt`.

b. **Race Condition** terjadi ketiga 2 atau lebih thread berlomba-lomba mengubah data secara bersamaan secara simultan, dan hasilnya bergantung pada urutan eksekusi instruksi yang tidak dapat diprediksi / Non-Deterministik. Race Condition ini disebabkan karena adanya tidak adanya mekanisme sinkronisasi yang menjaga critical region.

3. Perbaiki program ini agar tidak terjadi race condition lagi!

Solusi perbaikan dari program ini, kita harus memastikan adanya Mekanisme Sinkronisasi agar hanya 1 thread yang berada / mengakses Critical Region pada 1 waktu. Cara paling umum adalah dengan menggunakan mekanisme MUTEX (Mutual Exclusion).

```
// SAVE DENGAN NAMA : prak6-1-mod_NIM.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NITTERS 10000000 // Jumlah iterasi yang akan dilakuka
n oleh setiap thread
void *count(void *arg);
unsigned int cnt = 0; // Variabel global yang akan digunakan
sebagai counter bersama
// variabel cnt ini akan diakses oleh kedua thread
pthread_mutex_t lock; // 1. Deklarasi MUTEX

int main() {
        // 2. Inisialisasi MUTEX
        pthread_mutex_init(&lock, NULL);

    // Deklarasi thread
    pthread_t tid1, tid2;

    // Membuat thread
    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);

    // Menunggu kedua thread selesai
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    // Mengecek nilai counter setelah kedua thread selesai
    if(cnt != (unsigned)NITTERS * 2) {
```

```
        printf("Wrong.. cnt = %d\n", cnt);
    } else {
        printf("Ok.. \tcnt = %d\n", cnt);
    }
    pthread_mutex_destroy(&lock); // 3. Hapus mutex setelah s
elesai
    exit (0);
}

// Function yang akan dijalankan oleh kedua thread untuk mela
kukan increment pada cnt
void *count(void *arg) {
    int i;

    for(i = 0; i < NITTERS; i++) {
            pthread_mutex_lock(&lock); // 4. Kunci sebelum ma
suk critical region
        cnt++;
        pthread_mutex_unlock(&lock); // 5. Buka kunci setelah
selesai
    }
}
```

C. PRACTICUM 6.2 :

1. prak6-2 : MUTEX & Busy-Waiting/Polling

Modifikasi program prak6-1_NIM.c menggunakan MUTEX Facility dari pthread library, dan save dengan nama prak6-2_NIM.c. Berikut beberapa ketentuannya :

a. Tambahkan variabel MUTEX GLOBAL dengan deklarasi berikut:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

b. Kunci Critical Region pada program dengan menambahkan instruksi berikut:

```
pthread_mutex_lock(&mutex);
..... CRITICAL REGION .....
pthread_mutex_unlock(&mutex);
```

c. Kode program tersebut akan seperti berikut:

```
// SAVE DENGAN NAMA : prak6-2-mutex_NIM.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NITTERS 10000000

void *count(void *arg);
unsigned int cnt = 0;

// Inisialisasi global mutex secara statis
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main() {
    pthread_t tid1, tid2;

    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    if(cnt != (unsigned)NITTERS * 2) {
        printf("Wrong.. cnt = %u\n", cnt);
    } else {
        printf("Ok.. \tcnt = %u\n", cnt);
    }
```

```
    exit(0);
}

void *count(void *arg) {
    int i;
    for(i = 0; i < NITTERS; i++) {
        // --- LOCK (Awal Critical Region) ---
        pthread_mutex_lock(&mutex);

        cnt++; // Critical Region

        // --- UNLOCK (Akhir Critical Region) ---
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

d. Compile dengan perintah `gcc prak6-2-mutex_NIM.c -o prak6-2-mutex_NIM -pthread`

2. PENJELASAN prak6-2 :

a. **Observasi apakah program sesuai dengan nilai yang anda harapkan / perhitungan matematis?**

Ya, sekarang program akan selalu menghasilkan output yang benar. Hal ini terjadi karena mutex menjamin sifat Mutual Exclusion, dimana hanya satu thread yang diizinkan melakukan operasi `cnt++` pada satu waktu. Tidak ada lagi iterasi yang "hilang" akibat pembacaaan data yang interleaving / tumpang tindih.

b. **Apakah program lebih lambat saat dijalankan?**

Ya, secara signifikan program akan terasa lebih lambat saat dijalankan. Hal ini terjadi karena beberapa faktor yaitu:

- Overhead pada Fungsi, dimana setiap iterasi / perulangan (10 Juta per thread) kini harus memanggil fungsi `lock` dan `unlock`.
- Serialisasi, proses yang terjadi sebelumnya berjalan secara paralel (Bersamaan), kini dipaksa berjalan bergantian secara serial untuk mengakses

`cnt`.

- Context Switching, dimana CPU harus bekerja lebih keras mengelola antrean thread yang menunggu giliran memegang kunci MUTEX.

3. SKENARIO pada Operasi `lock` & `unlock` :

a. Apa yang terjadi jika `pthread_mutex_lock()` dijalankan 2 kali berturut-turut?

Jika sebuah thread memanggil lock pada mutex yang sudah ia kunci sendiri (tanpa ada unlock diantaranya), maka akan terjadi Deadlock (self-Deadlock). Hal yang akan anda observasi:

- Thread tersebut akan berhenti (Hang / Freeze) selamanya.
- Thread menunggu kunci dilepaskan, padahal yang memegang kuncinya adalah dirinya sendiri yang sedang dalam kondisi menunggu.

b. Apa yang terjadi jika `pthread_mutex_unlock()` tidak disertakan?

Hal yang akan terjadi adalah Deadlock antar thread. Hal yang akan anda amati nantinya:

- Thread pertama berhasil melakukan `lock` dan mengeksekusi satu kali `cnt++`.
- Karena tidak ada `unlock`, kunci tetap dipegang oleh thread pertama.
- Thread kedua akan tertahan selamanya saat mencoba melakukan `lock`.
- Thread pertama pun akan tertahan selamanya saat mencoba melakukan `lock` diiterasi kedua.

Hasilnya program akan freeze dan tidak akan pernah selesai.

4. PERBEDAAN MUTEX & Semaphore:

a. Konsep Dasar MUTEX & Semaphore

- MUTEX adalah mekanisme locking, ibarat sebuah kunci toilet tunggal, hanya orang yang memegang kunci yang bisa masuk, dan hanya orang yang sama yang bisa membuka kuncinya setelah selesai.
- Semaphore adalah mekanisme signaling. Ibarat sebuah tempat parkir dengan jumlah slot tertentu. Jika ada slot kosong, kendaraan boleh masuk. Jika penuh,

harus menunggu sampai ada kendaraan lain keluar dan memberi signal bahwa slot tersedia.

b. Perbedaan Utama MUTEX & Semaphore :

- MUTEX (`pthread_mutex_t`) :
  - OWNERSHIP : MUTEX memiliki konsep Ownership, thread yang mengunci harus yang menjadi thread yang membuka kunci juga.
  - VALUE : Hanya memiliki 2 status yaitu Terkunci (0) atau Terbuka (1).
  - TUJUAN UTAMA : Menjamin Mutual Exclusion pada Critical Region.
- Semaphore (`sem_t`) :
  - OWNERSHIP : Tidak ada owneship, Satu thread bisa menunggu (`wait`), dan thread lain bisa mengirim sinyal (`post`)
  - VALUE : Memiliki nilai Integer (0, 1, 2, ...). Jika nilainya > 1 maka disebut sebagai Counting Semaphore, kalau hanya terdiri dari 2 nilai yaitu 0 dan 1 maka disebut sebagai binary semaphore.
  - TUJUAN UTAMA : Mengatur akses ke sejumlah RESOURCE atau sinkronisasi antar thread.

c. FENOMENA "PRIORITY INVERSION" pada MUTEX

Salah satu perbedaan teknis yang krusial dalam MUTEX adalah mengenai Priority Inversion pada MUTEX. MUTEX sering kali memiliki protocol priority inheritance. Jika thread prioritas rendah memegang kunci dan thread prioritas tinggi menunggu, sistem akan "Meminjamkan" prioritas tinggi ke thread rendah agar ia cepat selesai dan melepas kunci.

5. IMPLEMENTASI Busy-Waiting atau Polling Method dalam Lock Testing :

a. Implementasi Busy-Waiting atau Polling untuk Pengujian Lock : Base Program prak6-1_NIM.c

- Buat program baru dengan nama prak6-2-busy_NIM.c dengan menggunakan kode berikut:

```
while ((ret=pthread_mutex_trylock(&mutex)) == EBUSY);
```

- tambahkan file header `#include <errno.h>`
- Jalankan kedua program, dan observasi dengan perintah `top`, mana yang lebih banyak menggunakan CPU?
- Jika anda menghapus pthread_mutex_unlock() pada versi Busy-Waiting (`prak6-2-busy_NIM.c`) apakah program masih tetap berjalan? apakah program akan selesai?

b. Implementasi Program prak6-2-busy_NIM.c

```
// SAVE DENGAN NAMA : prak6-2-busy_NIM.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <errno.h> // Diperlukan untuk konstanta EBUSY

#define NITTERS 10000000

void *count(void *arg);
unsigned int cnt = 0;

// Inisialisasi mutex
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main() {
    pthread_t tid1, tid2;

    printf("Memulai thread dengan metode Busy-Waiting...\n");

    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);
```

```
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    if(cnt != (unsigned)NITTERS * 2) {
        printf("Wrong.. cnt = %u\n", cnt);
    } else {
        printf("Ok.. \tcnt = %u\n", cnt);
    }

    return 0;
}

void *count(void *arg) {
    int i, ret;

    for(i = 0; i < NITTERS; i++) {
        /**
         * BUSY-WAITING / POLLING LOOP
         * pthread_mutex_trylock akan langsung kembali (non-b
locking).
         * Jika mutex sedang dikunci thread lain, ia mengemba
likan EBUSY.
         * Loop while ini akan terus berputar memakan siklus
CPU sampai
         * mutex berhasil didapatkan (return 0).
         */
        while ((ret = pthread_mutex_trylock(&mutex)) == EBUS
Y);

        // Critical Region
        cnt++;

        // Melepas kunci
        pthread_mutex_unlock(&mutex);
    }
```

```
    return NULL;
}
```

c. Observasi dengan perintah `top`

Jalankan top pada terminal yang berbeda dengan kedua program tersebut dijalankan. Anda akan melihat perbedaan yang drastis dibandingkan `prak6-2_NIM.c` yang standar:

- **Versi Standar (`pthread_mutex_lock`):** Saat menunggu, thread akan "tidur" (*blocked*). Penggunaan CPU akan terlihat efisien karena thread tidak melakukan komputasi apa pun saat menunggu antrean.
- **Versi Busy-Waiting (`trylock`):** Thread tidak pernah tidur. Ia terus melakukan loop `while` dengan kecepatan jutaan kali per detik hanya untuk mengecek status kunci. Ini akan menyebabkan penggunaan CPU melonjak hingga **100%** pada core yang digunakan

d. Jika anda menghapus pthread_mutex_unlock() pada versi Busy-Waiting (`prak6-2-busy_NIM.c`) apakah program masih tetap berjalan? apakah program akan selesai?

- **Apakah program akan tetap berjalan?**

Ya, program akan tetap berjalan (running), tetapi tidak akan melakukan progress apapun.

- **Apakah program akan selesai?**

Tidak, program tidak akan pernah mencapai baris `printf()` atau `exit`

- Fenomena apa yang akan anda alami?

Fenomena ini adalah deadlock, khusunya Livelock. Berbeda dengan deadlock standar dimana thread “tidur” selamanya, pada Livelock thread tetap aktif menggunakan sumber daya CPY (karena loop `while` terus berjalan), namun ia terjebak dalam kondisi dimana statusnya tidak pernah berubah. Thread tersebut sibuk melakukan “Sesuatu” (Polling) tetapi sesuatu yang dikerjakan tidak membuahkan hasil.

D. PRACTICUM 6.3 :

1. prak6-3_NIM.c

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NITERS 100000000

pthread_mutex_t mutex_1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex_2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_1(void *arg);
void *thread_2(void *arg);

int main()
{
    pthread_t tid1, tid2;

    pthread_create(&tid1, NULL, thread_1, NULL);
    pthread_create(&tid2, NULL, thread_2, NULL);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
}

void *thread_1(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++)
    {
        pthread_mutex_lock(&mutex_1);
```

```
        pthread_mutex_lock(&mutex_2);

        printf("Thread 1\n");

        pthread_mutex_unlock(&mutex_2);
        pthread_mutex_unlock(&mutex_1);
    }
}

void *thread_2(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++)
    {
        pthread_mutex_lock(&mutex_2);
        pthread_mutex_lock(&mutex_1);

        printf("Thread 2\n");

        pthread_mutex_unlock(&mutex_1);
        pthread_mutex_unlock(&mutex_2);
    }
}
```

Program ini akan mengalami DEADLOCK. Output mungkin hanya akan mencetak beberapa baris “Thread 1” atau “Thread 2”, lalu berhenti sepenuhnya tanpa pernah selesai. Hal ini terjadi karena adanya fenomena Circular Wait, dimana kita harus menentukan urutan penguncian:

- Thread 1 mengunci `mutex_1` lalu mencoba mengunci `mutex_2`.
- Thread 2 mengunci `mutex_2` lalu mencoba mengunci `mutex_1`.

Jika Thread 1 sudah memegang `mutex_1` dan pada saat yang hampir bersamaan Thread 2 sudah memegang `mutex_2`, maka keduanya akan saling menunggu kunci yang dipegang pihak lain selamanya.

Ibaratnya :

- **Thread 1** berkata: "Saya ambil kunci 1 dulu, baru kunci 2."
- **Thread 2** berkata: "Saya ambil kunci 2 dulu, baru kunci 1."

Ini ibarat dua orang di gang sempit yang berpapasan. Orang pertama ingin lewat ke kanan, orang kedua ingin lewat ke kiri. Keduanya saling mengunci jalan masing-masing dan tidak ada yang mau mengalah.

Contoh lagi :

Bayangkan sebuah transaksi bank antar dua rekening.

- **Kunci 1 (`mutex_1`)** adalah akses ke Rekening A.
- **Kunci 2 (`mutex_2`)** adalah akses ke Rekening B.

Jika Anda ingin memindahkan uang dari A ke B, Anda harus mengunci **keduanya** agar tidak ada thread lain yang mengubah saldo A atau B di tengah proses transfer. Jika Anda hanya mengunci satu, ada risiko data tidak konsisten pada rekening lainnya.

2. Instruksi dalam Critical Region & Cara Menghindari Deadlock

Critical Regioin pada Program ini adalah baris kode yang berada diantara penguncian pertama dan pembukaan kunci terakhir.

```
printf("Thread 1 \n"); // Atau "Thread 2"
```

Ada beberapa aturan emas untuk menghindari deadlock:

1. **Lock Ordering (Urutan Penguncian):** Pastikan semua thread mengunci mutex dengan urutan yang sama (misal: semua thread harus kunci `mutex_1` dulu baru `mutex_2`).
2. **Lock Timeout:** Menggunakan `pthread_mutex_timedlock`. Jika tidak bisa dapat kunci dalam waktu tertentu, lepaskan semua kunci dan coba lagi nanti.
3. **Trylock & Back-off:** Menggunakan `pthread_mutex_trylock`. Jika gagal mengunci kunci kedua, thread harus melepas kunci pertama agar tidak menghalangi thread lain.

3. Ganti prak6-3_NIM.c menggunakan Semaphore

TIPS UBAH DARI MUTEX Menjadi semaphore :

1. Ganti atau tambahkan header `<semaphore.h>` untuk mengakses fungsi-fungsi semaphore.
2. Ganti variabel `pthread_mutex_t` menjadi `sem_t`. Untuk program Anda yang menggunakan dua kunci, deklarasinya menjadi:

```
sem_t sem1;
sem_t sem2;
```

3. Inisialisasi (Di dalam `main`)

Berbeda dengan Mutex yang bisa diinisialisasi secara statis dengan `PTHREAD_MUTEX_INITIALIZER`, Semaphore (unnamed) harus diinisialisasi menggunakan fungsi `sem_init()`.

Format Fungsi: `sem_init(pointer_sem, pshared, value);`

- **pshared**: Isi `0` jika hanya digunakan antar thread dalam satu proses.
- **value**: Isi `1` untuk menjadikannya **Binary Semaphore** (berfungsi seperti Mutex). Value awal.

```
sem_init(&sem1, 0, 1);
sem_init(&sem2, 0, 1);
```

4. Mengganti Fungsi Lock & Unlock, ini adalah inti dari perubahannya. Padanan fungsinya adalah :

- `sem_wait(&s);`
- `sem_post(&s);`

```
// SAVE DENGAN NAMA : prak-6-3-Deadlock_NIM.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
#include <semaphore.h>
#include <fcntl.h>

#define NITERS 1000000

// Deklarasi pointer untuk named semaphores
sem_t *sem1, *sem2;

void *thread_1(void *arg) {
    for (int i = 0; i < NITERS; i++) {
        sem_wait(sem1); // Lock sem1
        sem_wait(sem2); // Lock sem2
        printf("Thread 1 \n");
        sem_post(sem2); // Unlock sem2
        sem_post(sem1); // Unlock sem1
    }
    return NULL;
}

void *thread_2(void *arg) {
    for (int i = 0; i < NITTERS; i++) {
        sem_wait(sem2); // Lock sem2 (Potensi Deadlock karena
urutan beda)
        sem_wait(sem1); // Lock sem1
        printf("Thread 2 \n");
        sem_post(sem1); // Unlock sem1
        sem_post(sem2); // Unlock sem2
    }
    return NULL;
}

int main() {
    pthread_t tid1, tid2;

    // Inisialisasi Named Semaphore
    sem1 = sem_open("/sem_satu", O_CREAT, 0644, 1);
```

```
    sem2 = sem_open("/sem_dua", O_CREAT, 0644, 1);

    pthread_create(&tid1, NULL, thread_1, NULL);
    pthread_create(&tid2, NULL, thread_2, NULL);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    // Tutup dan hapus semaphore
    sem_close(sem1);
    sem_close(sem2);
    sem_unlink("/sem_satu");
    sem_unlink("/sem_dua");

    return 0;
}
```

```
// SAVE DENGAN NAMA : prak6-3-NoDeadlock_NIM.c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <fcntl.h>

#define NITERS 100000 // Jumlah iterasi dikurangi agar output
terlihat cepat

// Deklarasi pointer untuk named semaphores
sem_t *sem1, *sem2;

void *thread_1(void *arg) {
    for (int i = 0; i < NITERS; i++) {
        // URUTAN: SEM1 -> SEM2
        sem_wait(sem1);
```

```
        sem_wait(sem2);

        if (i % 10000 == 0) printf("Thread 1 sedang bekerj
a...\n");

        sem_post(sem2);
        sem_post(sem1);
    }
    printf("--- Thread 1 SELESAI ---\n");
    return NULL;
}

void *thread_2(void *arg) {
    for (int i = 0; i < NITERS; i++) {
        // PERBAIKAN: URUTAN DISAMAKAN DENGAN THREAD 1 (SEM1
-> SEM2)
        // Sebelumnya di kode deadlock, ini adalah sem2 -> se
m1
        sem_wait(sem1);
        sem_wait(sem2);

        if (i % 10000 == 0) printf("Thread 2 sedang bekerj
a...\n");

        sem_post(sem2);
        sem_post(sem1);
    }
    printf("--- Thread 2 SELESAI ---\n");
    return NULL;
}

int main() {
    pthread_t tid1, tid2;

    // Inisialisasi Named Semaphore dengan nilai awal 1 (Bina
ry Semaphore)
```

```
    // O_CREAT: Buat jika belum ada, 0644: Izin akses, 1: Nil
ai awal
    sem1 = sem_open("/sem_satu", O_CREAT, 0644, 1);
    sem2 = sem_open("/sem_dua", O_CREAT, 0644, 1);

    // Cek jika gagal buka semaphore
    if (sem1 == SEM_FAILED || sem2 == SEM_FAILED) {
        perror("sem_open gagal");
        exit(1);
    }

    pthread_create(&tid1, NULL, thread_1, NULL);
    pthread_create(&tid2, NULL, thread_2, NULL);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    // Tutup dan hapus semaphore dari sistem
    sem_close(sem1);
    sem_close(sem2);
    sem_unlink("/sem_satu");
    sem_unlink("/sem_dua");

    printf("Program selesai dengan sukses tanpa deadloc
k.\n");
    return 0;
}
```

E. PRACTICE :

```
// SAVE DENGAN NAMA : exercise_philosopher_NIM.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
#include <stdlib.h>

#define NUM_PHIL 5 // Jumlah Philosopher
#define SLEEP_TIME 2 // Sleep Time

pthread_mutex_t forks[NUM_PHIL];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left_fork = id; // Garpu Kiri
    int right_fork = (id + 1) % NUM_PHIL; // Garpu Kanan

    while (1) {
            // Filosofer berpikir dengan waktu random / sleep
random
        printf("Philosopher %d is thinking.\n", id);
        sleep(rand() % SLEEP_TIME);
                // Filosofer bangun dan akan masuk ke State L
apar
        printf("Philosopher %d is HUNGRY.\n", id);
                // Mencoba kunci garpu kiri
        pthread_mutex_lock(&forks[left_fork]);
        printf("Philosopher %d picked up left fork %d.\n", i
d, left_fork);
                // Menunggu 1 detik jika berhasil dapat garpu
kiri
        sleep(1);
                // Mencoba kunci garpu kanan
        pthread_mutex_lock(&forks[right_fork]);
        printf("Philosopher %d picked up right fork %d and is
EATING.\n", id, right_fork);
                // Makan selama waktu acak
        sleep(rand() % SLEEP_TIME);
                // Lepas garpu
        pthread_mutex_unlock(&forks[right_fork]);
        pthread_mutex_unlock(&forks[left_fork]);
```

```
        printf("Philosopher %d finished eating and put down f
orks.\n", id);
    }
}

int main() {
    pthread_t phils[NUM_PHIL];
    int phil_ids[NUM_PHIL];

    for (int i = 0; i < NUM_PHIL; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }

    for (int i = 0; i < NUM_PHIL; i++) {
        phil_ids[i] = i;
        pthread_create(&phils[i], NULL, philosopher, &phil_id
s[i]);
    }

    for (int i = 0; i < NUM_PHIL; i++) {
        pthread_join(phils[i], NULL);
    }

    return 0;
}
```

1. Masalah Utama :

Kode ini memiliki celah fatal yang disebut **Deadlock**. Karena adanya `sleep(1)` setelah mengambil garpu kiri, skenario berikut sangat mungkin terjadi:

1. Filosof 0 mengambil garpu 0.
2. Filosof 1 mengambil garpu 1.
3. Filosof 2 mengambil garpu 2.

4. Filosof 3 mengambil garpu 3.
5. Filosof 4 mengambil garpu 4.

Hasilnya: Setiap filosof sekarang memegang satu garpu di tangan kiri dan menunggu selamanya agar garpu di tangan kanan mereka (yang sedang dipegang oleh tetangga sebelah kanan) dilepaskan. Program akan berhenti mencetak apa pun karena semua thread "macet".

2. Solusi Paling Mudah :

Salah satu cara termudah adalah dengan membuat salah satu filosof (misalnya filosof terakhir) mengambil garpu kanan terlebih dahulu baru kiri, atau menggunakan aturan bahwa filosof hanya boleh mengambil garpu jika kedua garpu tersedia (menggunakan `pthread_mutex_trylock`).

3. Resource Hierarchy (Hierarki Sumber Daya)

Strategi ini sangat sederhana: Kita mengubah urutan pengambilan garpu untuk **salah satu filosof saja** (biasanya filosof terakhir).

- **Filosof 0, 1, 2, 3:** Mengambil garpu **Kiri** dulu, baru **Kanan**.
- **Filosof 4 (Terakhir):** Mengambil garpu **Kanan** dulu, baru **Kiri**.

Dengan cara ini, lingkaran "saling menunggu" (*Circular Wait*) akan terputus.

💡 <u>SOLUTION PRACTICE 6</u>